动态规划

qwaszx

2026 年 9 月 18 日

前言

动态规划（Dynamic Programming，DP）是一种强大的算法设计方法，可以在问题仅仅具有较弱的结构时得到较为高效的解决方案。动态规划的基本思想是通过合适的方式压缩表示和逐步生成所有的合法对象，并在过程中计算合法对象的某些信息（如最大权值、数量等）。寻找这种合适的表示和生成方式是动态规划的核心，也是最具挑战性的部分。

本讲义将介绍动态规划的基本概念和方法，并通过例子来展示动态规划的设计思想与优化方法。

目录

第一章 常见组合结构上的 DP **3**

1.1 序列前缀/线性 DP 3
1.2 序列区间 4
1.3 括号序列 4
1.4 排列 4
1.5 背包/卷积 6
1.6 树 8
1.6.1 树上背包 8
1.6.2 换根 10
1.6.3 树路径问题 10
1.7 数位 11
1.8 指数级状态 11
1.8.1 排列 11
1.8.2 集合与子集 12
1.8.3 生成树 14
1.9 网格/轮廓线 14

第二章 数据结构优化 DP **15**

2.1 空间优化 15
2.2 数据结构优化填表 15
2.2.1 求和型 15
2.2.2 最值型 16
2.3 整体 DP 16
2.4 矩阵乘法 16
2.5 凸包 17
2.5.1 凸集、凸包、凸函数 17
2.5.2 二维凸包 19

第三章 决策单调性 **22**

3.1 基本概念和应用 22
3.2 Monge 性质 22

参考文献 **24**

1 常见组合结构上的 DP

问题的组合结构往往提示了我们应该按照何种方式逐步生成对象。在根据组合对象确定子结构后，还需要考虑其他需要的信息、扫描的顺序等。一般来说，可以枚举各种扫描的顺序，然后按照“缺什么补什么”的方式设计状态。

1.1 序列前缀/线性 DP

序列的一个自然子结构是前缀。许多情况下，在序列上 DP 时考虑的一个维度都是前 i 个元素，然后通过向后扩展一个或者一段元素来转移。也可以用填表来理解，这样就是考虑最后一个元素的归属。

例 1.1. 经典问题.

- 最长上升/不降子序列的长度/个数
- 最大子段和
- 最长公共子序列

练习 1.2. P2679.

例 1.3. 给定一个序列 a，需要对其进行一些修改来得到一个单调不降的序列 b。分别最小化

- $\sum_i [b_i \neq a_i]$
- $\sum_i |b_i - a_i|$

解答. 对于第一问，答案为 n 减去最长不降子序列的长度。

对于第二问，首先可以说明存在一种最优方案，其所有 b_i 都来自 a 中的某个元素。否则，考虑所有最优方案中来自 a 中的元素最多的那个方案，其中必然存在某个 b_i 不属于 a。考虑 b_i 所在的极长等值段，我们必然可以让这个等值段 $+1$ 或者 -1，同时代价不变。我们可以不断地做这个操作，直到 b_i 变得属于 a，这就得到了一个来自 a 中元素更多的方案，矛盾。

那么我们就得到一个直接的 dp：设 $f_{i,j}$ 表示已经构造了 b 的前 i 个元素，最后一个元素是 j 的最小代价，其中 j 只需要考虑 a 中的元素。

更进一步. 第二问有一个贪心做法：初始每个元素都视为单独的一段，每段内 b 都取该段内 a 的中位数（如有多个就取最小的那个）。如果存在相邻两段不满足单调性条件，就将其合并。重写系统的 Newman 引理可以说明任意合并顺序得到的结果都是相同的，因此可以从左到右合并，使用可并堆维护即可。

该算法的正确性可以这样证明：算法执行的任意时刻，都存在一个全局最优解 b'，使得当前的每个段内 b' 都相同。考虑合并两个段之前的某个这样的最优解 b'，考虑 b' 在被合并的两个段上的值，我们总可以调整这两个值使它们相等，同时不增加代价，这就得到了合并后也符合要求的最优解。

评注. 这两个问题都是**保序回归**问题的一个特例。

练习 1.4. 要求变成 b 单调递增。提示

例 1.5. [CEOI 2005] Mobile Service.

1.2　序列区间

序列的另一个自然子结构是区间。相比前缀，区间允许更复杂的嵌套结构。括号序列、合并类问题、从序列构造树等类型的问题常常会使用区间 DP 来处理。

常见的区间 DP 类型有：

- 约简后的合法状态具有区间分解结构，即可以分解为若干不交子区间的并（也可能再加上一些剩余元素）
- 通过在两侧添加元素来扩展区间

可以考虑最后一个/第一个的情况来导出区间 DP 结构。

例 1.6. [NOI1995] 石子合并.

1.3　括号序列

在学习组合数学时，我们知道以下组合对象的结构都是类似的：

- 括号序列
- ± 1 序列
- 折线图
- 儿子有顺序的二叉树/多叉树
- 三角剖分

这几种组合对象的结构可以帮助我们进行 DP。

例 1.7. 给定一个由 $+1$，-1，？组成的序列，对每个 k，询问有多少种把？替换成 $+1$ 或 -1 的方法，使得最大前缀和恰好为 k。

1.4　排列

根据排列的生成方式，往往会考虑以下几种方式：

- 按照下标逐个放置元素
- 按照下标逐个**插入**元素，即考虑最后插入的值的排名，如 1324 变成 14253
- 逐个放置每个值
- 逐个**插入**每个值

按值生成也可以视为在逐下标确定 π^{-1}。在问题具有特殊结构的情况下，有时可以一次扩展多个元素，用数学方法确定这部分元素的方案数。

例 1.8. 经典问题. 给一个序列 a_i，问有多少个排列 π 满足 $\forall i,\ \pi_i \le a_i$.

评注. 一种直观化排列的方式是匹配。

例 1.9. [AtCoder DP Contest] Permutation. 给定一个由 $<$ 和 $>$ 组成的长度为 $n-1$ 的序列，问有多少个长为 n 的排列 π 满足这个关系。

解答. 按照下标逐个插入元素。在状态中记录结尾元素的排名即可。

例 1.10. [CSP-S 2025] 员工招聘.

解答. 按下标逐个放置元素。状态中还需要记录前面未录用的人数 j 以及 $c > j$ 的人中还没放的人数 k。然而，这样不能直接转移，因为无法知道下一个位置不录用时再下一个位置的候选人数会如何变化：假设目前不录用人数是 j，而之前可能已经录用了一些 $c = j+1$ 的人，但这个人数没有在状态中记录。

解决方法是所谓的“贡献延后计算”：我们不具体放置元素，而是只标记放置的元素是否 $> j$。等到 j 变化到 $j+1$ 时，再确定 $c = j+1$ 的人在前面具体是如何放置的。

另一个视角. 假设我们先钦定了所有位置的录取情况，那么如何计算方案数？对于一个录取的位置，必须有 $s_i = 1$ 且 $c_{\pi_i} > f_i$，其中 f_i 为 i 前面的未录取人数；对于一个未录取的位置，需要 $s_i = 0$ 或者 $c_{\pi_i} \le f_i$。那么相当于每个位置有限制 $c_{\pi_i} > f_i$ 或者 $c_{\pi_i} \le f_i$ 或者无限制。

现在来考虑子问题：(i) 如果只有某一种限制怎么做？(ii) 如果只有某两种限制怎么做？(iii) 怎么把某一种限制用另外两种限制表示？

如果把无限制表示成 $[c_{\pi_i} > f_i] + [c_{\pi_i} \le f_i]$，那就能得到前面的做法。如果把 $[c_{\pi_i} > f_i]$ 表示成 $1 - [c_{\pi_i} \le f_i]$，那么就得到另一种做法，也就是容斥。

最后，可以在 dp 中枚举所有位置的录取情况。

例 1.11. [JOI Open 2016] 摩天大楼. 求有多少个 a 的排列满足相邻位置差的绝对值之和 S 不超过 L。

解答. 逐个放置每个值与逐个插入每个值的结合。具体来说，在逐个插入每个值的过程中，还需要钦定哪些相邻元素在最终的排列中也是相邻的。换句话说，在插入的过程中维护若干连续段，每一段都代表最终排列中的一个连续段。

从小到大考虑每个值。转移分为三种：添加一个单点作为新段、将一个点添加到某个段的一端、添加一个点并粘合两个段。注意连续段分为两种：贴着边界的（只有一个端点）和不贴着边界的（有两个端点），因此还需要记录目前有 0/1/2 个段已经贴着边界了。

这样插入每个值时，其对 S 的贡献是确定的。但是对这个题而言还需要一个拆分贡献的技巧：把 $a_j - a_i$ 拆成 $\sum_{k=i+1}^{j} a_k - a_{k-1}$，也就是把两个数造成的贡献拆到它们之间每个数插入的时候计算。

例 1.12. [ARC148E]　K. 求有多少种 a 的排列使得相邻位置总满足 $a_i + a_{i+1} \ge K$。

解答. 构造一个插入顺序，使得每个数都满足以下二者之一：

- 后面所有的数都可以和它相邻
- 后面所有的数都不能和它相邻

这样在逐个插入每个数的过程中维护可用的空位数即可，一个数的任何合法插入方式都会得出相同的空位数。

顺序可以这样构造：先排序，令 $l=1$，$r=n$。每次检查是否 $a_l+a_r\geq K$，如果是则将写下 a_r 并让 $r\leftarrow r-1$，否则写下 a_l 并让 $l\leftarrow l+1$。

评注. 略微加强版 *P5606* 小 K 与毕业旅行

例 1.13. CF1437F Emotional Fishermen.

解答. 如果序列的前缀 max 已经确定，那么我们可以直接算出将剩余的值插入进排列的方案数。对前缀 max DP 的过程中计算其他数的贡献即可。

在只关心排列的“区间最值”结构时，通常会用笛卡尔树来刻画排列。这时，往往会使用区间 DP 来处理。

例 1.14. ARC183C Not Argmax.

例 1.15. CF1580B Mathematics Curriculum.

1.5　背包/卷积

许多问题的组合结构是卷积的形式：体积 a 和体积 b 的对象组合起来得到体积 $a+b$ 的对象。由于“体积”是额外的结构，它常常会与其他结构结合起来。

背包的常见技巧包括

- 生成函数
- 换维

例 1.16. [SCOI2009] 粉刷匠.

例 1.17. CF1425B Blue and Red of Our Faculty!. 转化后变为计算

$$\sum_i \sum_{k\in[l_i,r_i]} [x^k]\prod_{j\neq i}(x^{w_j}+x^{-w_j}+1),$$

其中 $\sum_j w_j=n$

例 1.18. CF1442D Sum.

例 1.19. [十二省联考 2019] 皮配. 有两种限制，因此使用两个维度，x 代表阵营，y 代表派系。由于总数固定，一个维度上的限制相当于限制了上下界。若选手选择蓝色则在 x 一维贡献 s_i 的次数，否则贡献 0 的次数；选手选择鸭系则在 y 一维贡献 s_i 的次数，否则贡

献 0 的次数。
对于一个城市，设其学校总重量为 ss，特殊学校的集合为 T。对于一个特殊学校 i，令 $c_{i,p,q}$ 表示此学校能否选择 (红 (0)/蓝 (1))(R(0)/鸭 (1)) 的组合，$F_{i,p}(y) = c_{i,p,0} + c_{i,p,1}y^{s_i}$，则该城市的方案数可以被生成函数刻画为

$$\prod_{i\notin T}(1+y^{s_i})\left(\sum_{i\in T}F_{i,0}(y)+x^{ss}\sum_{i\in T}F_{i,1}(y)\right).$$

总的生成函数就是所有城市的生成函数的乘积。显然，每个城市是两部分 $A(y)$ 和 $B(x,y)$ 的乘积，可以分开计算 $\prod A(y)$ 和 $\prod B(x,y)$。由于 T 的总大小不超过 k，$\prod B(x,y)$ 在 y 上的次数不超过 ks，可以 $O(k\cdot M\cdot ks)$ 计算 $\prod B(x,y)$。$\prod A$ 可以直接 $O(nm)$ 计算。最后可以 $O(ksM)$ 枚举 $\prod B(x,y)$ 的每一项，$O(1)$ 计算这一项与 $\prod A$ 的乘积对答案的贡献。

例 1.20. 同余背包. 给定若干个数 a_i，求 $[1,M]$ 内有多少个数可以被这些数完全背包得到。

对于最优化问题，还有一些特殊的结果成立。以下都只考虑重量为整数的背包，体积（或者说重量）用 w 表示，价值用 v 表示。

引理 1.21. 对于任意要求总体积不超过 W 的背包问题，考虑按照性价比贪心选择直到第一次放不下得到的方案 G，则存在一个最优方案与 G 的差别（异或意义）不超过 $2w_{\max}$ 个物品。

证明. 不妨认为是 01 背包，不是 01 背包可以展开成 01 背包。任取一种最优方案 O，假设它与 G 的差别超过 $2w_{\max}$ 个物品。可以通过从 G 中删除 $G\backslash O$ 中的物品并添加 $O\backslash G$ 中的物品来得到 O。按照“小了就加大了就删”的原则进行这个过程，可以使得过程中的体积总落在 $[W-w_{\max},W+w_{\max}]$ 之间。那么由于这个过程的长度超过 $2w_{\max}$，必然存在两个时刻的体积相同。也就是说这两个时刻之间加入和删除的物品总体积相同，而 $G\backslash O$ 在性价比排序中处在 $O\backslash G$ 的前面，从而将这两个时刻之间的过程删除后不会更劣。如此反复可以调整出一个差别不超过 $w_{\max}$ 的最优方案。 □

练习 1.22. 对于完全背包，证明这个界可以为 $w_{\max}-1$。

练习 1.23. 说明也存在一种最优方案与 G 的差别不超过 $2v_{\max}$ 个物品。

推论 1.24. 背包问题可以做到 $O(n\min(V^2,W^2))$。

引理 1.25. 对于完全背包，总体积为 W 的一种最优方案一定可以由某对 x,y 的最优方案加起来得到，其中 $|x-y|\le w_{\max}$。

证明. 考察任何一种总体积为 W 的最优方案，可以将其分成两部分，使得两部分的体积差不超过 $w_{\max}$。这可以排序后按奇数位置和偶数位置分块得到。也就是说 $\mathrm{ans}_W \le \max_{x+y=W,|x-y|\le w_{\max}}(\mathrm{ans}_x+\mathrm{ans}_y)$。

反过来，x 的一种方案和 y 的一种方案可以拼成 $x+y$ 的一种方案，于是 $\text{ans}_{x+y} \geq \text{ans}_x + \text{ans}_y$。这就完成了证明。□

推论 1.26. 存在 $O(w_{\max}^2 \log w_{\max})$ 的完全背包算法。

证明. 由引理 1.21，可以先将 W 降低至 $w^* \cdot w_{\max}$ 级别。

由引理 1.25，可以倍增出答案：维护 $[W - w_{\max}, W + w_{\max}]$ 范围内的答案即可 $O(w_{\max}^2)$ 计算出 $[2W - w_{\max}, 2W + w_{\max}]$ 的答案。

所以可以做到 $O(w_{\max}^2 \log w_{\max})$ 的复杂度。□

例 1.27. 体积和价值相同的 01 背包。$O(nw_{\max})$

解答. 首先按照引理 1.21 随便选出一个方案 G，而存在体积为 T 的方案当且仅当存在一种选择 $U \subseteq G$ 和 $V \subseteq \overline{G}$ 的方案，使得按照“小了就加大了就删”原则从 G 中按顺序删除 U 并加入 V 时体积总在 $[T - W, T + W]$ 范围内，且最终体积为 T。

考虑对这种东西 DP。设 $f_{i,j,k}$ 表示考虑了 G 中前 i 个物品和 $\bar{G}$ 中前 j 个物品时，是否存在一个 $U \subset G_{1,\dots,i}$ 和 $V \subseteq (\bar{G})_{1,\dots,j}$ 按前述原则执行后能够得到体积 k。这样是 $O(n^2 w_{\max})$ 的，可以用 bitset 进一步加速。

注意到在 k 固定时，$f_{\cdot,\cdot,k}$ 为 1 的部分是一条轮廓线。也就是说存在一个不增的序列 $b_k(i)$ 使得 $f_{i,j,k}$ 为 1 当且仅当 $j \geq b_k(i)$。我们转而计算这些 b_k。将 f 上的转移对应到 b 上：

1. $f_{i,j,k} \to f_{i+1,j,k}$ 对应 $b_k(i) \to b_k(i+1)$
2. $f_{i,j,k} \to f_{i,j+1,k}$ 在 b 中无意义
3. $f_{i,j,k} \to f_{i+1,j,k-G_{i+1}} (k \geq T)$ 对应 $b_k(i) \to b_{k-G_{i+1}}(i+1)$
4. $f_{i,j,k} \to f_{i,j+1,k+(\bar{G})_{j+1}} (k < T)$ 对应 $j \to b_{k+(\bar{G})_{j+1}}(i) (j \geq b_k(i))$

显然只有最后一条的转移不是 $O(1)$ 的。但是注意到第四条实际上只需要对 $b_k(i) \leq j < b_k(i-1)$ 的那些 j 执行，因为 $j \geq b_k(i-1)$ 的部分可以由 $i-1$ 的第四条转移再加上 $i-1$ 的第一条转移得到。这样每个 k 在整个过程中执行的转移数量就不超过 $O(n)$。

练习 1.28. [THUPC 2023 初赛] 背包.

1.6　树

树上的常见子结构是子树与从根出发的链。

例 1.29. 经典问题. 求树上最大权独立集/最小权点覆盖/直径。

1.6.1　树上背包

首先我们说明如果每个点的体积都是 $O(1)$，那么在只枚举必要范围转移时，树上背包的复杂度和序列背包相同。

定理 1.30. 如果每个点的体积都是 $O(1)$，那么无体积限制的树上背包复杂度是 $O(n^2)$ 的。

证明. 每个点对只在 LCA 处贡献一次复杂度。 □

定理 1.31. 如果每个点的体积都是 $O(1)$，体积上限为 V，那么树上背包复杂度是 $O(nV)$ 的。

证明. 首先将树变成二叉树以简化分析，算法中不需要真的这么做。将点按照是否满足 $\text{size}_u \le V$ 分开。

$\text{size}_u \le V$ 的结点构成若干不交子树，这部分贡献的复杂度为每棵子树根的 size_u^2，总共不超过 nV。

把 $\text{size}_u > V$ 的结点称为重结点。对于重结点，考虑几种情况：

- 至多一个儿子满足 $\text{size}_v \le V$。此时这里贡献至多 $\text{size}_v \cdot V$ 的复杂度，且不同重结点对应的 v 的子树不交，从而总共不超过 $O(nV)$。
- 两个儿子都满足 $\text{size}_v > V$。这样的点是重结点虚树中的点，由于重结点构成的连通块的叶子数量不超过 n/V，虚树中的点数不会超过 $2n/V$，每个点贡献 $O(V^2)$，总共 $O(nV)$。

□

除了常规的子树 DP 外，还有一种按 DFS 序 DP 的树上背包算法，该算法可以在点体积**不为 1** 时也做到 $O(nV)$ 的复杂度，缺点是只能计算整棵树的答案而无法计算每个子树内的答案。

算法 1.32. DFS 序树上背包. 按 DFS 序考虑点，将树上背包的限制视为：若想要选择区间 $(\text{dfn}_u, \text{dfn}_u + \text{size}_u)$ 中的点，则必须选择 dfn_u。从后往前 DP，设 $f_{i,j}$ 表示考虑了 $\text{dfn}_u \ge i$ 的点、总体积为 j 的答案。转移时，根据 $\text{dfn}_u = i$ 的点是否选择递归到子问题 $f_{i+1,j+v_u}$ 或 $f_{i+\text{size}_u,j}$。

例 1.33. [HAOI2015] 树上染色. 选择 k 个点染黑，其他点染白，最大化同色点对距离和。

例 1.34. [USACO02FEB] 重建道路. 删掉尽量少的边，使得分离出来的森林中存在一个大小为 k 的连通块。

练习 1.35. [JSOI2016] 最佳团体. 找到性价比最高的恰有 k 个点的连通块，点集 S 的性价比定义为 $\frac{\sum_{i\in S} a_i}{\sum_{i\in S} b_i}$。

练习 1.36. P1270 “访问”美术馆. 寻找一个最大的权值不超过 V 的连通块，连通块的权值定义为点权之和加上二倍的边权之和。

练习 1.37. [JSOI2018] 潜入行动.

例 1.38. [IOI 2005] Riv 河流.

练习 1.39. 选一个 k 个点的点集 S，最小化 $\sum_i w_i d(i,S)$，其中 $d(i,S) = \min_{u\in S} d(i,u)$。树有非负边权。

练习 1.40. 在上一题中把 $\sum$ 换成 max。这样会有一个贪心做法。

例 1.41. [HEOI2013] SAO. 给一棵树，每条边有一个 $<$ 或者 $>$ 表示顺序要求，求合法排列的数量。

1.6.2　换根

树形 DP 通常可以算出有根树的每个子树的答案。之后，可以通过“换根”技巧计算出每个子树补的答案，以及以每个点为根时整棵树的答案。

例 1.42. 子树补. 设 S_u 为 u 的子树，$\overline{S_u}$ 为 u 的子树补（或者说以 u 为根时，原 fa_u 对应子树），v 为 u 的任一儿子，则 $\overline{S_v} = \overline{S_u} + (S_u \backslash S_v)$。因此，可以通过 $\overline{S_u}$ 的信息和 $S_u \backslash S_v$ 的信息组合出 $\overline{S_v}$ 的信息。
$S_u \backslash S_v$ 的信息通常有以下几种计算方法：

- 直接减掉（加和类信息）、利用次小/次大值（最值类信息）
- 通过前后缀信息拼合
- 线段树分治

练习 1.43. 利用子树补信息计算以每个点为根时整棵树的信息。

练习 1.44. [POI 2008] STA-Station. 对每个点计算所有点到它的距离和。

练习 1.45. CF543D Road Improvement.

练习 1.46. CF512D Fox And Travelling.

1.6.3　树路径问题

在状态中记录子树内端点个数即可拼接出路径。

例 1.47. CF1146F Leaf Partition.

解答. 考虑子树的情况，不难发现有以下几种可能：

- 子树的叶子划分中没有任何一组的 f 包含 u，且未来也不会包含
- 子树的叶子划分中没有任何一组的 f 包含 u，但是未来会包含
- 子树的叶子划分中有恰好一组的 f 包含 u

三种情况的方案数分别记为 $f_{u,0/1/2}$。如果令 (u, v) 这条边包含在某一组对应的连通块中，那么贡献为 $f_{v,1/2}$；否则，贡献为 $f_{v,0/2}$。具体转移如下：

- 对于 $f_{u,0}$，每个儿子都需要是 $f_{v,0/2}$，不同儿子分到不同组
- 对于 $f_{u,1}$，选择一个 $f_{v,1/2}$ 继续往上延伸，其他儿子都取 $f_{v,0/2}$
- 对于 $f_{u,2}$，选择至少两个 $f_{v,1/2}$，其他儿子都取 $f_{v,0/2}$

最后根的答案是 $f_{u,0/2}$，叶子的初值是 $f_{u,2} = 1, f_{u,0/1} = 0$。

例 1.48. [SCOI2015] 小凸玩密室.

练习 1.49. [COCI 2014/2015 #1] Kamp.

练习 1.50. [COCI 2020/2021 #2] Svjetlo.

1.7　数位

数位 DP 的基本模式是在状态中假设高位的一些情况（如目前是否贴着上界、是否还没有出现首位等），计算低位有多少种方案。

有时也可以直接枚举和上界的 LCP（最长公共前缀）与下一位，然后计算低位任取的所有方案中有多少个合法。这种方式也可以视为在 Trie 上 DP。

练习 1.51. P4317 花神的数论题. 计算 $\prod_{i=1}^{n} \mathrm{popcount}(i)$（对质数取模）。

练习 1.52. CF855E Salazar Slytherin's Locket. 多次询问：$[l,r]$ 中有多少个数在 b 进制下满足 $0,1,\cdots,b-1$ 的出现次数都是偶数。
尽量让复杂度与 b 无关。

练习 1.53. P13085 [SCOI2009] windy 数. 计算 $[l,r]$ 中有多少个数满足相邻数位上的数字之差都至少为 2。

例 1.54. P2481 [SDOI2010] 代码拍卖会. 计算有多少个数位上的数字不降的 n 位数模 P 为 0。

例 1.55. P3303 [SDOI2013] 淘金. 求 $[l,r]$ 内的所有数的代价和，一个数的代价定义为其各个数位的带权中点。

例 1.56. U117267 特.

例 1.57. P7961 [NOIP2021] 数列.

例 1.58. P7156 [USACO20DEC] Cowmistry P.

1.8　指数级状态

1.8.1　排列

在状态中不记录排列的顺序，而是只记录存在的值来将状态数从 $n!$ 降至 2^n。

例 1.59. 挑战 NP Hard. 求最长哈密顿路/哈密顿回路；计算积和式；拓扑序计数

例 1.60. P6622 [省选联考 2020 A/B 卷] 信号传递.

例 1.61. P2157 [SDOI2009] 学校食堂.

1.8.2　集合与子集

首先我们给出集合操作的若干常用技巧。

代码 1.1: 枚举子集

```
// 枚举 S 的非空子集 T
for(int T=S;T;T=(T-1)&S){
}
```

练习 1.62. 写出枚举超集的算法。

算法 1.63. 高维前缀和/差分. 给定 f_S，计算 $\hat{f}_S = \sum_{T\subseteq S} f_T$。
或者反过来，给定 $\hat{f}_S$，还原出 f_S。

算法 1.64. 按位或卷积. 需要计算

$$h_C = \sum_{A\cup B=C} f_A g_B.$$

首先计算

$$\begin{aligned}\hat{h}_C &= \sum_{A\cup B\subseteq C} f_A g_B \\ &= \sum_{A\subseteq C} f_A \sum_{B\subseteq C} g_B \\ &= \hat{f}_C \hat{g}_C,\end{aligned}$$

于是 $O(n2^n)$ 计算出 $\hat{f}, \hat{g}$，点积得到 $\hat{h}$，再做高维差分还原出 h 即可。

练习 1.65. 计算按位与卷积，即

$$h_C = \sum_{A\cap B=C} f_A g_B.$$

练习 1.66. 不做高维差分还原 h，而是直接利用 $\hat{f}$ 和 $\hat{g}$ 在 $O(2^n)$ 时间内计算 h 的一项。

算法 1.67. 子集卷积. 需要计算

$$h_C = \sum_{A\uplus B=C} f_A g_B.$$

关键在于

$$[A \uplus B = C] = [A \cup B = C][|A| + |B| = |C|].$$

于是使用生成函数转换为

$$\sum_{A\cup B=C} f_A x^{|A|} g_B x^{|B|},$$

可以 $O(n^2 2^n)$ 计算。

算法 1.68. $(\max,+)$ **位运算卷积.** 需要计算

$$h_C = \max_{A\cup B=C}(f_A + g_B).$$

分治，变成计算

$$h_0 = f_0 * g_0, h_1 = (f_0 * g_1) \oplus (f_1 * (g_0 \oplus g_1)).$$

复杂度 $T(n) = 3T(n-1) + O(2^n) = O(3^n)$。注意每层递归只需要 2^{n-1} 的额外空间，于是可以做到总共 2^n 额外空间。

代码 1.2: $(\max,+)$ 位运算卷积

```
void solve(int* a, int* b, int* c, int* tmp, int n) {
        if (n == 1)
    {
        c[0]=max(c[0],a[0]+b[0]);
        return;
    }
        solve(a, b, c, tmp, n >> 1);
        solve(a, b + (n >> 1), c + (n >> 1), tmp, n >> 1);
        rep(i, 0, (n >> 1) - 1) tmp[i] = b[i], b[i] = max(b[i], b[i + (n
           >> 1)]);
        solve(a + (n >> 1), b, c + (n >> 1), tmp + (n >> 1), n >> 1);
        rep(i, 0, (n >> 1) - 1) b[i] = tmp[i];
}
```

评注. 也有一个直接算的做法。枚举 A，就有

$$h_C = \max_{A\subseteq C}\left(f_A + \max_{C-A\subseteq B\subseteq C} g_B\right).$$

考虑计算所有

$$\hat{g}(U,V) = \max_{S\subseteq V} g_{U\cup S},$$

对于每个 U 在 $2^{n-|U|}$ 时间内递推计算出所有 $\hat{g}(U,V)$，总共 $O(3^n)$。于是

$$h_C = \max_{A\subseteq C} f_A + \hat{g}(C-A, A).$$

这样需要 3^n 空间。可以先枚举 $C-A$，然后再枚举 C 和 A，这样只需要同时计算一个 U 的 $\hat{g}(U,V)$，空间变成 $O(2^n)$，但是转移代价会多一个 n。

例 1.69. 挑战 NP Hard. 集合覆盖问题。带权/无权

练习 **1.70. P2831** [**NOIP 2016** 提高组] 愤怒的小鸟.

例 **1.71. P2150** [**NOI2015**] 寿司晚宴.

1.8.3　生成树

例 **1.72. P3959** [**NOIP 2017** 提高组] 宝藏.

例 **1.73.** 最小斯坦纳树.

1.9　网格/轮廓线

例 **1.74. P2704** [**NOI2001**] 炮兵阵地.

例 **1.75. P4363** [九省联考 **2018**] 一双木棋 **chess**.

2 数据结构优化 DP

2.1 空间优化

由于绝大多数情况下 DP 都是在一个 DAG 上转移，可以对此 DAG 分层，每次只保留当前层以及转移需要的层来优化空间复杂度。

例 2.1. 滚动数组. 若 $f[i][]$ 的转移只需要 $i-k \le j < i$ 的 j，则只需要存储这 $k+1$ 层（有时是 k 层）的状态，据此将第一维从 n 优化至 k。

如果问题还需要输出方案，那么我们可以通过分治或者分块的方式还原方案。

例 2.2. 滚动数组输出方案. P7685 [CEOI 2005] Mobile Service

分块. 按 B 分块，记录第一维在所有 kB 位置的 DP。每次计算一块的方案，首先重算这 B 个位置的 DP 结果，然后倒着算出这一块的方案。据此将第一维的空间从 n 优化至 $O(\sqrt{n})$。

分治. $\mathrm{solve}(l, r, S_l, S_r)$ 表示已知一个最优解在 l 时状态为 S_l，在 r 时状态为 S_r，需要还原这段区间的方案。首先进行一遍 DP 得到这个最优解在 $m = (l+r)/2$ 时刻的状态 S_m，然后递归两边即可。

可以将第一维的空间从 n 优化至 $\log n$，但是时间往往需要多一个 $\log n$。

2.2 数据结构优化填表

本节中，我们考虑最常见的一维 DP 填表转移 $f_i = \bigodot_{j\in[l_i,r_i]} g(j,i)$。对于多维的情况，其思想也是适用的。

在绝大多数情况下，这本质上是一个**数据结构问题**：我们要查询一个范围中 $g(j,i)$ 的加和，然后计算出 f_i，再考虑 f_i 对后续的影响。例如：

- $g(j,i)$ 可以分解为 j 的贡献 a_j 与 i 的贡献 b_i，此时计算 f_i 相当于查询 $\bigodot_{j\in[l_i,r_i]} a_j$，然后根据 f_i 计算出 a_i。
- 将 DP 视为关于 i 的扫描线，在 $i \to i+1$ 时维护 $g(\cdot, i)$。

2.2.1 求和型

优化 $f_i = \sum_{j\in[l_i,r_i]} g(j,i)$ 类型的 DP。根据 l_i, r_i 和 g 的性质，可以使用不同的数据结构优化。

例 2.3. 设 $g_{j,i}$ 可以拆成 $O(1)$ 个 $a_i b_j$ 之和，则可以对 b 做前缀和来 $O(1)$ 查询。

练习 2.4. 每个 i 向 $[l_i, r_i]$ 连有向边，保证 $i < l_i \le r_i$。求 1 到每个点的路径数量。

例 2.5. 设 $g_{j,i}$ 可以拆成 $O(1)$ 个 $a_ib_jc_{i-j}$ 之和，且 $[l_i, r_i]$ 与 $[1, i)$ 的差别只有 $O(1)$ 项，则可以使用半在线卷积计算，此时 $O(n \log^2 n / \log\log n)$ 是一个常见的复杂度。

2.2.2　最值型

优化 $f_i = \max_{j\in[l_i,r_i]} g(j,i)$ 类型的 DP。根据 l_i, r_i 和 g 的性质，可以使用不同的数据结构优化。由于失去可可减性，这类问题通常比求和型问题更加困难。

练习 2.6. 考虑 $g(j,i) = a_j + b_i$ 的形式。

- 使用双端队列在 $O\left(\sum_i |l_i - l_{i-1}| + |r_i - r_{i-1}|\right)$ 复杂度内维护。
- 使用 RMQ 来 $O(1)$ 查询 f_i。

例 2.7. P1848 [USACO12OPEN] Bookshelf G.

练习 2.8. 每个 i 向 $[l_i, r_i]$ 连带权有向边，保证 $i < l_i \le r_i$。求 1 到每个点的最长路长度。

2.3　整体 DP

有时我们需要处理一些多维的 DP，其中某一维上的转移基本都是原地且相同的。对于这种问题，我们可以考虑直接使用数据结构的区间/全局操作来维护这一维。

例 2.9. P9400 「DBOI」Round 1 三班不一般. 支持以下转移：

- 对 $0 < j < A$，$f_{i,j} = a_i f_{i-1,j-1} + b_i$
- $f_{i,0} = a_i \sum_j f_{i-1,j}$

例 2.10.

2.4　矩阵乘法

有时，DP 转移可以视为一个矩阵乘法。此时可以使用矩阵快速幂来询问远处点值，或者用数据结构维护矩阵乘法乘积来支持修改或区间询问。

评注. 对于询问远处点值的情况，矩阵快速幂通常可以被一个线性递推替代。

例 2.11. P6772 [NOI2020] 美食家.

例 2.12. P4719 【模板】动态 DP.

2.5　凸包

2.5.1　凸集、凸包、凸函数

定义 2.13. 凸组合. 对于 $\mathbb{R}^d$ 中的若干个点 $\mathbf{x}_i$ 的线性组合 $\sum_i \theta_i \mathbf{x}_i$，若系数 θ 满足 $\sum_i \theta_i = 1$ 且 $\theta_i \geq 0$，则称此线性组合为一个**凸组合**。

定义 2.14. 凸集. 对于点集 $S \subseteq \mathbb{R}^d$，如果其中任何两个点 $\mathbf{x}_1, \mathbf{x}_2$ 的所有凸组合 $\theta\mathbf{x}_1 + (1-\theta)\mathbf{x}_2$ 都在 S 中，则称 S 是凸的。换句话说，S 是凸的当且仅当连接其中任意两点的线段都被包含在 S 中。

性质 2.15.

- 凸集的交是凸集
- 凸集的 Minkowski 和 $\{\mathbf{x}+\mathbf{y} : \mathbf{x} \in S_1, \mathbf{y} \in S_2\}$ 是凸集
- 凸集的直和 $\{(\mathbf{x}, \mathbf{y}) : \mathbf{x} \in S_1, \mathbf{y} \in S_2\}$ 是凸集

定义 2.16. 凸包. 对于点集 $S \subseteq \mathbb{R}^d$，其**凸包**（也称**凸壳**）定义为包含 S 的最小凸集，即所有包含 S 的凸集的交集。

定理 2.17. 设 $S \subseteq \mathbb{R}^d$。以下表述等价：

1. C 是 S 的凸包
2. C 是 S 中元素的所有有限凸组合构成的集合，即

$$\left\{ \sum_{i=1}^{k} \theta_i \mathbf{x}_i : \theta_i \geq 0, \sum_i \theta_i = 1, \mathbf{x}_i \in S \right\}.$$

3. C 是 S 中所有 $d+1$ 个点的凸组合构成的集合

若 S 有限，则以下表述等价：

1. C 是 S 的凸包
2. C 是包含 S 的最小凸多面体
3. C 是所有包含 S 的半超平面的交

事实 2.18. 点 $\mathbf{p}$ 是 S 的凸包顶点当且仅当它不是 $S - \{\mathbf{p}\}$ 中点的凸组合。因此，对于有限点集 S，凸包的顶点数量不超过 $|S|$。

引理 2.19. 对于有限点集 $S \subseteq \mathbb{R}^d$，令 C 为其凸包。对于任何 $\mathbf{w} \in \mathbb{R}^d$，都有

$$\max_{\mathbf{x} \in S} \mathbf{w} \cdot \mathbf{x} = \max_{\mathbf{x} \in \partial C} \mathbf{w} \cdot \mathbf{x}.$$

证明. 由于 S 有限，C 是闭集，故最大值可以在某个 $x \in \mathbf{S}$ 中取到。对于 $\mathbf{x} \in \mathrm{int}(C)$，必然可以将其向某一方向移动 ϵ 距离，使得移动后仍然处于 C 中，且 $\mathbf{w} \cdot \mathbf{x}'$ 增大。故最大值只能在 ∂C 上取到。　□

在绝大多数情况下，我们只需要关注二维情况。此时，凸包是一个凸多边形。

定义 2.20. 上凸壳和下凸壳. 对于点集 $S \subseteq \mathbb{R}^2$，设 C 为其凸包。考虑 C 中 x 坐标最小和最大的点 L, R，顺时针沿着凸包边界从 L 走到 R 得到的路径称为**上凸壳**，从 R 走到 L 得到的路径称为**下凸壳**。

推论 2.21. 考虑点集 $S \subseteq \mathbb{R}^2$。对于给定的 k，$\max_{(x,y)\in S} kx + y$ 在上凸壳中取到，$\min_{(x,y)\in S} kx + y$ 在下凸壳中取到。$(x_0, y_0) \in S$ 处可以取得最值当且仅当直线 $y - y_0 = k(x - x_0)$ 是凸包的切线。

练习 2.22. 对 $\min_{(x,y)\in S} kx + y$ 给出类似的结论。

定义 2.23. 凸函数. 若定义在凸集 C 上的函数 $f : C \to \mathbb{R}$ 满足对任何 $\mathbf{x}_1, \mathbf{x}_2 \in C$ 和 $\theta \in (0, 1)$，都有

$$f(\theta\mathbf{x}_1 + (1-\theta)\mathbf{x}_2) \le \theta f(\mathbf{x}_1) + (1-\theta) f(\mathbf{x}_2)$$

成立，则称 f 是**凸 (convex)** 的。若不等号严格成立，则称 f 是**严格凸 (strictly convex)** 的。若不等式反向成立，则称 f 是**凹 (concave)** 的。

评注. 在国内的一些教材中，凸和凹的定义是反过来的。为了避免混淆，你也可以使用“上凸”和“下凸”来代替“凸”和“凹”。在本讲义中，我们遵循国际惯例，使用“凸”来表示“下凸”，使用“凹”来表示“上凸”。

评注. 在这个定义中，我们并不要求函数可导。

性质 2.24. 设 $f : C \to \mathbb{R}$ 是一个凸函数。有以下性质成立：

- 切线在图像下方：假设 f 在 $\mathrm{int}(C)$ 上可导。对任何 $\mathbf{x}_1, \mathbf{x}_2 \in \mathrm{int}(C)$，都有 $f(\mathbf{x}_2) \ge f(\mathbf{x}_1) + \nabla f(\mathbf{x}_1) \cdot (\mathbf{x}_2 - \mathbf{x}_1)$。反过来，满足该性质的函数是一个凸函数；若进一步满足不等号严格成立，则函数是严格凸的。
  - 对于一维情况，就是 $f(x_2) \ge f(x_1) + f'(x_1)(x_2 - x_1)$。
- 假设 f 在 $\mathrm{int}(C)$ 上二阶可导。对任何 $\mathbf{x}$，$\nabla^2 f(\mathbf{x})$ 是半正定的。反过来，满足该性质的函数是一个凸函数；若进一步满足不等号严格成立，则函数是严格凸的。
  - 对于一维情况，即 $f''(x) \ge 0$。这也可以理解为斜率不降：对任何 $x < y < z$，都有 $\frac{f(y)-f(x)}{y-x} \le \frac{f(z)-f(x)}{z-x} \le \frac{f(z)-f(y)}{z-y}$。
- f 的**上图像 (epigraph)** $\mathrm{epi}(f) = \{(\mathbf{x}, y) : y \ge f(\mathbf{x})\}$ 是凸集。反过来，满足该性质的函数是凸的。
- Jensen 不等式：对于任意 $\mathbf{x}_1, \ldots, \mathbf{x}_n \in C$ 的凸组合 $\overline{\mathbf{x}} = \sum_{i=1}^n \theta_i \mathbf{x}_i$，都有

$$f\left(\sum_{i=1}^n \theta_i \mathbf{x}_i\right) \le \sum_{i=1}^n \theta_i f(\mathbf{x}_i).$$

性质 2.25.

- 凸函数的非负线性组合是凸的
- 凸函数的 max 是凸的
- 凸函数复合一次函数是凸的
- 设 $f, g : \mathbb{R} \to \mathbb{R}$ 是凸的，且 g 在 $f(\mathbb{R})$ 上不降，则 $g \circ f$ 是凸的。

例 2.26.

- 一次函数 $ax + b$ 同时是凸的和凹的
- 开口向上的二次函数 $a(x - b)^2 + c$ 是凸的
- 绝对值函数 $|x - a|$ 是凸的

以上讨论的都是连续情况下的定义，离散情况下则并没有一个统一的定义。我们使用一个相对常用的定义：

定义 2.27. 离散凸函数. 对于函数 $f : \{l, l + 1, \ldots, r\} \to \mathbb{R}$，若其二阶差分

$$\Delta^2 f(x) = f(x + 2) - 2f(x + 1) + f(x)$$

非负，则称 f 是凸的。若 $\Delta^2 f(x) > 0$，则称 f 是严格凸的。

练习 2.28. 对于定义域包含 $\{l, \ldots, r\}$ 的连续凸函数 f，证明：f 限制到 $\{l, \ldots, r\}$ 上得到的函数是离散凸的。

性质 2.29. 设 $f : \{l, \ldots, r\} \to \mathbb{R}$ 的凸的。以下命题等价：

- f 是凸的
- f 的斜率不降：对任何 $l \le x < y < z \le r$，都有 $\frac{f(y)-f(x)}{y-x} \le \frac{f(z)-f(x)}{z-x} \le \frac{f(z)-f(y)}{z-y}$。
- f 的一阶差分 $\Delta f(x) = f(x + 1) - f(x)$ 不降
- 切线在函数下方：对任何 $l \le x, y \le r, \Delta f(x - 1) \le k \le \Delta f(x)$，都有 $f(y) \ge f(x) + k(y - x)$。
- 延拓到连续函数后凸：对每个 $i, i + 1$，用一次函数连接 $(i, f(i))$ 和 $(i + 1, f(i + 1))$ 得到的 $[l, r]$ 上的连续函数 $\tilde{f}$ 是凸的。

利用最后一个命题，我们可以在连续凸和离散凸之间互相转化。

2.5.2　二维凸包

本节中，我们介绍二维凸包的算法和相关应用。

引理 2.30. 考虑点集 S 的凸壳上的所有顶点，一个点 $\mathbf{x}$ 顺/逆时针方向的下一个点 $\mathbf{y}$ 是使得 $\mathbf{y} - \mathbf{x}$ 的极角最大/小的 $\mathbf{y} \in S$。

评注. 这里大/小的定义为：在所有可能的跨度不超过 π 的极角中，逆时针的最后一个/第一个。

算法 2.31. Andrew 算法. Andrew 算法从左往右分别构建出上、下凸壳。将所有点按照 (x, y) pair 排序。从左到右扫描加入点，维护所有线段的上、下包络，每次弹出末尾的一段点。排序后线性。

评注. 常常只需要上/下凸壳中的一个。

评注. 一般来说，不需要保留线段的中间点。在需要保留所有边界点时，可以将弹栈的条件从严格不等号改为非严格不等号。

算法 2.32. Graham 算法. Graham 算法按照极角序构建凸包。首先选择 (x, y) 按 pair 比较时最小的一个，该点必然在凸包上，且所有其他点都在以其为原点的 $(-\frac{\pi}{2}, \frac{\pi}{2}]$ 角度范围内。按照角度排序后逆时针构建凸包即可。

例 2.33. 维护平面上的点集 S，支持以下操作：

- 插入一个点 (x_i, y_i)。保证 x 不降。
- 给定 k，询问 $\max_{(x,y)\in S} kx + y$。

练习 2.34. 假设询问的 k 不增/不降，说明取到最值的点只会在凸包上单调移动。据此设计一个 $O(n)$ 的算法。

练习 2.35. 优化 $f_i = \min_{j<i} k_i \cdot a_j + b_j$ 类型的 DP。当 k_i 不降时，这被称为**斜率优化**。P3195 [HNOI2008] 玩具装箱

例 2.36. 在上一题中去掉 x 不降的限制。

解答. 平衡树维护凸包/李超树/离线对时间分治。李超树比较好写且快。

评注. 实际上还可以支持删除，这就是完全动态凸包问题。

例 2.37. P3309 [SDOI2014] 向量集. 维护序列 (x_i, y_i)，支持以下操作：

- 在末尾插入一个点 (x_i, y_i)。
- 给定 l, r, a, b，询问 $\max_{i\in[l,r]} ax_i + by_i$。

练习 2.38. 额外支持末尾删除

练习 2.39. 优化 $f_i = \min_{j\in[l_i,r_i]} k_i \cdot a_j + b_j$ 类型的 DP。

在很多情况下（如背包问题），我们会使用集合的卷积 $H_i = \cup_j (F_j + G_{i-j})$，其中 $S+T = \{x+y : x \in S, y \in T\}$。注意这相当于对 $F = \{(i, x) : x \in F_i\}$ 和 $G = \{(i, y) : y \in G_i\}$ 做 Minkowski 和得到 $H = \{(i, z) : z \in H_i\}$。记 $C(S)$ 为点集 S 的凸包，则有以下性质成立：

性质 2.40.

- $C(S \cup T) = C(C(S) \cup C(T))$

- $C(S+T)=C(S)+C(T)$

于是，对点集进行运算后再求凸包可以变为对点集的凸包进行运算。并集的凸包容易计算，而 Minkowski 和的凸包可以通过以下算法计算。

算法 2.41. 凸包的 Minkowski 和. 只需计算两个凸包顶点集合 C_1, C_2 的 Minkowski 和。令 $\mathbf{p}_1, \mathbf{p}_2$ 分别为 C_1, C_2 中按 (x, y) 比较时最小的点，则 $\mathbf{p}=\mathbf{p}_1+\mathbf{p}_2$ 必然为 S_1+S_2 中 (x, y) 最小的点。
从 $\mathbf{p}$ 出发逆时针逐渐构建新凸包的顶点，设目前的点为 $\mathbf{x}_1+\mathbf{x}_2$，下一个顶点 $\mathbf{x}_1'+\mathbf{x}_2'$ 必然是关于当前顶点极角最小的一个，而这只有两种可能：

- $\mathbf{x}_1'$ 是 $\mathbf{x}_1$ 在 C_1 中的下一个点，$\mathbf{x}_2'=\mathbf{x}_2$
- $\mathbf{x}_1'=\mathbf{x}_1$，$\mathbf{x}_2'$ 是 $\mathbf{x}_2$ 在 C_2 中的下一个点

在其中选择较小的一个即可。复杂度 $O(|C_1|+|C_2|)$。

评注. 可以发现，这相当于把两个凸包的边“按极角归并”。

考虑函数的 $(\min,+)$ 卷积与逐点 min 的上图像，有以下事实成立：

事实 2.42. 考虑定义在整数上的函数 f, g。

- 考虑其 $(\min,+)$ 卷积 $h(i)=\min_j f(j)+g(i-j)$。则 $\operatorname{epi}(h)=\operatorname{epi}(f)+\operatorname{epi}(g)$。
- 考虑其逐点最小值 $h(i)=\min(f(i), g(i))$。则 $\operatorname{epi}(h)=\operatorname{epi}(f)\cup\operatorname{epi}(g)$。

考虑由若干函数进行一系列逐点 min 和 $(\min,+)$ 卷积得到的函数 f。有时，我们可以说明 f 是凸的；有时，我们只需要 $\operatorname{epi}(f)$ 的凸包（如只需要计算 $F(x)=\min_i f(i)+ix$）。此时，我们可以利用前述的点集凸包的运算来进行优化。首先，我们有以下事实：

事实 2.43. $\operatorname{epi}(f)$ 的凸包边界等于 $\{(i, f(i))\}$ 的下凸壳。

于是，我们只需要维护过程中的所有下凸壳，对下凸壳求并集或 Minkowski 和即可。

例 2.44. 在线性时间内计算凸函数的 $(\min,+)$ 卷积。

练习 2.45. 在线性时间内计算凸函数的 $(\max,+)$ 卷积。提示

例 2.46. P5073 [Ynoi Easy Round 2015] 世上最幸福的女孩. 全局加，询问区间最大子段和。离线。

3 决策单调性

3.1 基本概念和应用

最常见的情况是一维动态规划的决策单调性。

定义 3.1. 一维决策单调性. 考虑 $f_i = \min_{j<i} A(j,i)$ 形式的一维动态规划。定义 i 的决策点为

$$\mathrm{opt}(i) = \min \arg\min_{j<i} A(j,i),$$

即取到最值的 j 中最小的一个。若 $\mathrm{opt}(i)$ 单调不降，则称该动态规划具有**决策单调性**。

对于 A 不依赖 f 的情况，可以使用分治来快速计算 f。

算法 3.2. 决策单调性分治. $\texttt{solve}(l,r,L,R)$ 表示当前需要计算 $[l,r]$ 内的 f_i，且已知决策点在 $[L,R]$ 内。令 $m = \lfloor (l+r)/2 \rfloor$，$O(R-L+1)$ 计算 f_m 的最优决策点 p，递归处理 $\texttt{solve}(l,m-1,L,p)$ 和 $\texttt{solve}(m+1,r,p,R)$ 即可。
考虑分治树的每一层。递归到下一层时，$\sum R-L+1$ 的增加量为该层的结点个数。初始的 $R-L+1=n$，这部分总共贡献 $O(n\log n)$；第 i 层增加的 2^i 个结点会贡献 $\log_2 n - i + 1$ 次，这部分总贡献 $O(n)$。于是总复杂度 $O(n\log n)$。

练习 3.3. 考虑算法过程中依次询问的所有 $A(j_k,i_k)$。证明 $\sum_k |j_k - j_{k-1}| + |i_k - i_{k-1}| = O(n\log n)$。

练习 3.4. 给定一个排列，将其划分为 k 段，每段的代价为该段的逆序对数。最小化总代价。在已知此问题具有决策单调性的情况下，设计 $O(nK\log^2 n)$ 的算法。

3.2 Monge 性质

我们考虑一种被称为 Monge 性质的凸性，

定义 3.5. Monge 矩阵. 对于矩阵 A，若其混合差分

$$\Delta_{xy} A(i,j) = A(i+1,j+1) - A(i+1,j) - A(i,j+1) + A(i,j)$$

非正，则称 A 为 Monge 矩阵。

引理 3.6. 以下命题等价：

- A 是 Monge 矩阵
- A 的行差分 $\Delta_x A(i,j) = A(i+1,j) - A(i,j)$ 关于列 j 不增
- A 的列差分 $\Delta_y A(i,j) = A(i,j+1) - A(i,j)$ 关于行 i 不增

- 四边形不等式：对任意 $i_1 \le i_2, j_1 \le j_2$，都有 $A(i_1, j_1) + A(i_2, j_2) \le A(i_1, j_2) + A(i_2, j_1)$

参考文献

ChatGPT 5.6 (Thinking high) 辅助完成了一些内容的撰写和审查。强烈推荐大家使用 Gemini Pro (AI Studio)、ChatGPT Plus 或以上水平的 AI 工具辅助学习。

OI Wiki.

YeahPotato. DP 题方法总汇. 2023.

洛谷的诸多 DP 题单（搜索“DP”或“动态规划”），此处不再一一列出。